Secretaría de Educación de Guanajuato

SEP Secretaría de Educación Pública

Universidad Tecnológica del Suroeste de Guanajuato

TSU EN TECNOLOGÍAS DE LA INFORMACIÓN ÁREA DESARROLLO DE SOFTWARE MULTIPLATAFORMA

Sistemas operativos

Prácticas de cisco capitulos:9-10

Profesor: Dr. Martín Cano Contreras.

Nombre del alumno: Antonio Arias Gutiérrez

Grupo: 3A TIDEM

Valle de Santiago, Gto. 10 de julio del 2022

Capítulo 9

Un **script** es un archivo que dentro de el contiene una serie de instrucciones que tienen como objetivo realizar una acción, donde este se escribe en un archivo de texto con extensión **.sh** y se ejecuta mediante la instrucción **./nombredelarchivo.sh**

La siguiente instrucción nos arrojara en la pantalla un mensaje que nos muestre “Hello , World!**”**

```
  GNU nano 2.2.6              File: test.sh

#!/bin/sh
echo "Hello ,World!"

                              [ Read 2 lines ]
^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text^T To Spell
```

El archivo se le debe de asignar los permisos para que se ejecute con el comando **chmod 777 test.sh** y se ejecuta con el comando **./test.sh**

```
sysadmin@localhost:~$ nano test.sh
sysadmin@localhost:~$ sh test.sh
Hello ,World!
sysadmin@localhost:~$ chmod 777 test.sh
sysadmin@localhost:~$ ./test.sh
Hello ,World!
```

El siguiente script tiene como objetivo mostrar en pantalla un mensaje de “hola mundo “ y debajo nos mostrara la fecha actual.

```
  GNU nano 2.2.6              File: test.sh                        Modified

#!/bin/sh
echo "Hello, World!"
echo  -n "the time is "
date

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text^T To Spell
```

Se oprime la letra Ctrl + O para poder guardar el archivo y se oprime enter para guardarlo con el nombre que elijamos.

```
  GNU nano 2.2.6              File: test.sh                        Modified

#!/bin/sh
echo "Hello, World!"
echo  -n "the time is "
date

File Name to Write: test.sh
^G Get Help        M-D DOS Format     M-A Append         M-B Backup File
^C Cancel          M-M Mac Format     M-P Prepend
```

Después presionamos ctrl + x para poder salir del editor nano.

```
  GNU nano 2.2.6              File: test.sh

#!/bin/sh
echo "Hello, World!"
echo  -n "the time is "
date

                          [ Wrote 4 lines ]
^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text^T To Spell
```

Se ejecuta el archivo /test.sh y el resultado es lo siguiente:

```
sysadmin@localhost:~$ ./test.sh
Hello, World!
the time is Sun Jul 10 19:42:17 UTC 2022
```

Las variables se pueden encontrar como en la siguiente instrucción donde la variable es **ANIMAL** y dentro de ella se guarda un valor el cual es **“penguin”** y se mandan llamar con el carácter **“$”**seguido del nombre de la variable.

```
#!/bin/bash

ANIMAL="penguin"
echo "May favorite animal is a $ANIMAL"
```

El resultado del script será el siguiente, donde se ve reflejado el contenido de la variable en la misma frase.

```
sysadmin@localhost:~$ nano animal.sh
sysadmin@localhost:~$ ./animal.sh
May favorite animal is a penguin
```

Las variables se pueden guardar con y sin comillas como es el siguiente caso, además de que se puede guardar en una variable nueva el contenido de otra variable.

```
#!/bin/bash

ANIMAL=penguin
SOMETHING=$ANIMAL
echo "My favorite animal is a $ANIMAL"
```

Y al ejecutarlo nos muestra lo mismo que con el script anterior.

```
sysadmin@localhost:~$ ./animal.sh
My favorite animal is a penguin
```

Otra manera de poder asignarle un valor a una variable, es que su valor sea la ejecución de un comando, como el siguiente caso, arrojara la función del comando pwd que arroja el directorio en el que nos encontramos.

```
#!/bin/bash
CURRENT_DIRECTORY=`pwd`
echo "You are in  $CURRENT_DIRECTORY "
```

Donde el resultado sería el siguiente y se aprecia que se ejecuto el comando dentro de un echo.

```
sysadmin@localhost:~$ ./pwd.sh
You are in  /home/sysadmin
```

Una manera de que el usuario ingrese datos es por medio del comando read, que lo que hace es que lee lo que se ingrese por el teclado y lo guarda en una variable.

```
#!/bin/bash

echo -n "What is your name? "
read NAME
echo "Hello $NAME!"
```

El resultado del script es el siguiente.

```
sysadmin@localhost:~$ nano read.sh
sysadmin@localhost:~$ chmod 777 read.sh
sysadmin@localhost:~$ ./rad.sh
-bash: ./rad.sh: No such file or directory
sysadmin@localhost:~$ ./read.sh
What is your name? Antonio
Hello Antonio!
```

El comando grep lo utilizamos para saber si un valor está dentro, en este caso si el valor que ingresamos se encuentra dentro del directorio que ordenamos nos arroja un cero y si no encuentra nada arroja un 1.

```
sysadmin@localhost:~$ grep -q root /etc/passwd
sysadmin@localhost:~$ echo $?
0
sysadmin@localhost:~$ grep -q slartibartfast /etc/passwd
sysadmin@localhost:~$ echo $?
1
```

Un condicional es una instrucción que se ejecuta si algo se cumple y si no se cumple se realiza otra acción. Un ejemplo es la siguiente acción.

```
#!/bin/bash

if grep -q root /etc/passwd; then

  echo root is in the password file

else

  echo root is missing from the password file

fi
```

Con la instrucción if se puede hacer varias comparaciones como es en este caso, ya que escribimos cualquier palabra y si podemos alguna que se especifico en la instrucción nos arroja una acción.

```
#!/bin/bash


if [ "$1" = "hello" ]; then

  echo "hello yourself"

elif [ "$1" = "goodbye" ]; then

  echo "nice to have met you"

  echo "I hope to see you again"

else

  echo "I didn't understand that"

fi
```

```
sysadmin@localhost:~$ ./if.sh
I didn't understand that
sysadmin@localhost:~$ ./if.sh hello
hello yourself
sysadmin@localhost:~$ ./if.sh goodbye
nice to have met you
I hope to see you again
```

La siguiente instrucción nos sirve para pasar lista.

```
#!/bin/bash

for NAME in Sean Jon Isaac David; do
  echo "Hello $NAME"
done

for S in *; do
  echo "Doing something to $S"
done
```

Con la siguiente instrucción while nos arrojara una serie de números hasta que llegue al límite de la condición, que en este caso es el 10.

```
#!/bin/bash

i=0
while [ $i -lt 10 ]; do
  echo $i
  i=$(( $i + 1))
done
echo "Done counting"
```

```
sysadmin@localhost:~$ ./while.sh
0
1
2
3
4
5
6
7
8
9
Done counting
```

Capítulo 10

10.2.1 Paso 1

Con el comando se listan las especificaciones con las que cuenta el equipo que se está utilizando.

```
sysadmin@localhost:~$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                16
On-line CPU(s) list:   0-15
Thread(s) per core:    1
Core(s) per socket:    1
Socket(s):             16
NUMA node(s):          1
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 63
Stepping:              2
CPU MHz:               2593.993
BogoMIPS:              5187.98
Virtualization:        VT-x
Hypervisor vendor:     VMware
Virtualization type:   full
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
L3 cache:              20480K
NUMA node0 CPU(s):     0-15
```

10.2.2 Paso 2

El comando cat /proc/cpuinfo arroja al usuario mucha más información sobre el dispositivo que se está trabajando, muestra información más detallada de los

```
sysadmin@localhost:~$ cat /proc/cpuinfo
processor       : 0
vendor_id       : GenuineIntel
cpu family      : 6
model           : 63
model name      : Intel(R) Xeon(R) CPU E5-2640 v3 @ 2.60GHz
stepping        : 2
microcode       : 0x43
cpu MHz         : 2593.993
cache size      : 20480 KB
physical id     : 0
siblings        : 1
core id         : 0
cpu cores       : 1
apicid          : 0
initial apicid  : 0
fpu             : yes
fpu_exception   : yes
cpuid level     : 15
wp              : yes
flags           : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov
pat pse36 clflush dts mmx fxsr sse sse2 ss syscall nx pdpe1gb rdtscp lm constant
_tsc arch_perfmon pebs bts nopl xtopology tsc_reliable nonstop_tsc cpuid pni pcl
mulqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_ti
```

10.2.3 Paso 3

La memoria **RAM** es u componente del equipo de trabajo. Por lo que el comando **free -m** proporciona el numero de memoria que se está usando en megabytes y para poder observarlos en gigabytes se utiliza el comando **free -g**, así como la

```
sysadmin@localhost:~$ free -m
             total       used       free     shared    buffers     cached
Mem:        193423      17044     176378          0       1690       3418
-/+ buffers/cache:      11934     181488
Swap:         2047          0       2047
sysadmin@localhost:~$ free -g
             total       used       free     shared    buffers     cached
Mem:           188         16        172          0          1          3
-/+ buffers/cache:         11        177
Swap:            1          0          1
sysadmin@localhost:~$
```

10.2.4 Paso 4

El comando **lspci** cumple con el objetivo de poder listar todos los dispositivos que están conectador a los puertos de la tarjeta madre, esto para informarnos qué y que se encuentra conectado

```
sysadmin@localhost:~$ lspci
00:00.0 Host bridge: Intel Corporation 440BX/ZX/DX - 82443BX/ZX/DX Host bridge (
rev 01)
00:01.0 PCI bridge: Intel Corporation 440BX/ZX/DX - 82443BX/ZX/DX AGP bridge (re
v 01)
00:07.0 ISA bridge: Intel Corporation 82371AB/EB/MB PIIX4 ISA (rev 08)
00:07.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01)
00:07.3 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
00:07.7 System peripheral: VMware Virtual Machine Communication Interface (rev 1
0)
00:0f.0 VGA compatible controller: VMware SVGA II Adapter
00:10.0 SCSI storage controller: LSI Logic / Symbios Logic 53c1030 PCI-X Fusion-
MPT Dual Ultra320 SCSI (rev 01)
00:11.0 PCI bridge: VMware PCI bridge (rev 02)
00:15.0 PCI bridge: VMware PCI Express Root Port (rev 01)
00:15.1 PCI bridge: VMware PCI Express Root Port (rev 01)
00:15.2 PCI bridge: VMware PCI Express Root Port (rev 01)
00:15.3 PCI bridge: VMware PCI Express Root Port (rev 01)
00:15.4 PCI bridge: VMware PCI Express Root Port (rev 01)
00:15.5 PCI bridge: VMware PCI Express Root Port (rev 01)
00:15.6 PCI bridge: VMware PCI Express Root Port (rev 01)
00:15.7 PCI bridge: VMware PCI Express Root Port (rev 01)
00:16.0 PCI bridge: VMware PCI Express Root Port (rev 01)
00:16.1 PCI bridge: VMware PCI Express Root Port (rev 01)
```

10.2.5 Paso 5

El comando **lspci -k** lo utilizamos para poder visualizar los módulos utilizados, además para ver los dispositivos y el controlador del kernel.

```
sysadmin@localhost:~$ lspci -k
00:00.0 Host bridge: Intel Corporation 440BX/ZX/DX - 82443BX/ZX/DX Host bridge (
rev 01)
        Subsystem: VMware Virtual Machine Chipset
        Kernel driver in use: agpgart-intel
00:01.0 PCI bridge: Intel Corporation 440BX/ZX/DX - 82443BX/ZX/DX AGP bridge (re
v 01)
00:07.0 ISA bridge: Intel Corporation 82371AB/EB/MB PIIX4 ISA (rev 08)
        Subsystem: VMware Virtual Machine Chipset
00:07.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01)
        Subsystem: VMware Virtual Machine Chipset
        Kernel driver in use: ata_piix
00:07.3 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
        Subsystem: VMware Virtual Machine Chipset
00:07.7 System peripheral: VMware Virtual Machine Communication Interface (rev 1
0)
        Subsystem: VMware Virtual Machine Communication Interface
        Kernel driver in use: vmw_vmci
00:0f.0 VGA compatible controller: VMware SVGA II Adapter
        Subsystem: VMware SVGA II Adapter
        Kernel driver in use: vmwgfx
00:10.0 SCSI storage controller: LSI Logic / Symbios Logic 53c1030 PCI-X Fusion-
MPT Dual Ultra320 SCSI (rev 01)
        Subsystem: VMware LSI Logic Parallel SCSI Controller
```

10.2.6 Paso 6

Con la instrucción **lsusb** nos permite observar todos los dispositivos USB que se encuentran.

Pero en este caso no aparecen los dispositivo USB ya que se esta realizando en una maquina virtual por lo que no detecta ninguno.

```
sysadmin@localhost:~$ lsusb
unable to initialize libusb: -99
```

Ya que si lo estuviéramos realizando en una maquina normal el resultado sería el siguiente.

```
sysadmin@localhost:~$ lsusb
Bus 001 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
sysadmin@localhost:~$
```

10.2.7 Paso 7

lsmod se utilizó para poder visualizar los diferentes módulos que se encuentran instalados por medio del kernel, donde observamos cual es le nombre del modulo y la cantidad de memoria que utiliza ese modulo.

```
sysadmin@localhost:~$ lsmod
Module                  Size  Used by
netlink_diag           16384  0
vsock_diag             16384  0
sctp_diag              16384  0
sctp                  315392  117 sctp_diag
dccp_diag              16384  0
dccp                   73728  1 dccp_diag
tcp_diag               16384  0
udp_diag               16384  0
raw_diag               16384  0
inet_diag              24576  5 sctp_diag,dccp_diag,tcp_diag,udp_diag,raw_diag
unix_diag              16384  0
xt_nat                 16384  114
veth                   16384  0
xt_CHECKSUM            16384  1
iptable_mangle         16384  1
ipt_MASQUERADE         16384  6
nf_nat_masquerade_ipv4    16384  1 ipt_MASQUERADE
nf_conntrack_netlink    40960  0
nfnetlink              16384  2 nf_conntrack_netlink
```

10.2.9 Paso 9

El comando **fdisk -cul** muestra en pantalla los diferentes dispositivos de disco que se encuentran en el dispositivo, al igual que las particiones que tiene cada uno .

```
sysadmin@localhost:~$ fdisk -cul
Disk /dev/sda: 21.5 GB, 21474836480 bytes
255 heads, 63 sectors/track, 2610 cylinders
Units = cylinders of 16065 * 512 = 8225280 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x000571a2

   Device Boot      Start         End      Blocks   Id  System
/dev/sda1   *           1        2481    19921920   83  Linux
/dev/sda2            2481        2611     1046529    5  Extended
/dev/sda5            2481        2611     1046528   82  Linux swap / Solaris
```